

6 Synchronization

Background

- **Processes can execute concurrently**
  - May be interrupted at any time, partially completing execution

- **Concurrent access to shared data may result in data inconsistency**

- **Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes**

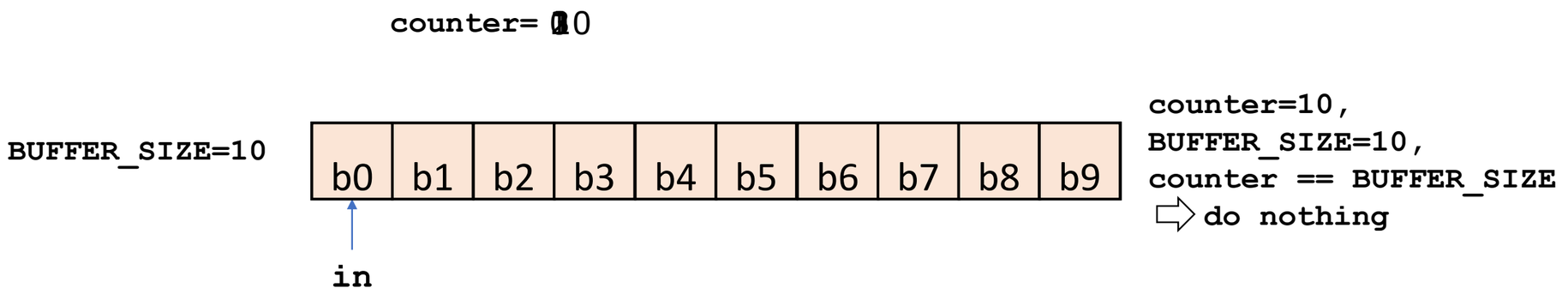
- **Illustration of the problem:**

  Suppose that we wanted to provide a solution to the consumer-producer problem that fills *all* the buffers. We can do so by having an integer `counter` that keeps track of the number of full buffers. Initially, `counter` is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

Producer

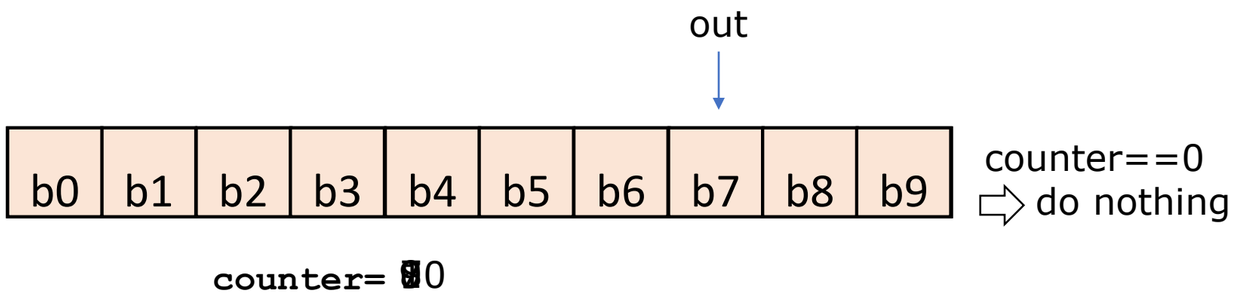
```
while (true) {
     /* produce an item in next_produced */

     while (counter == BUFFER_SIZE)
          ; /* do nothing */
     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```



Consumer

```
while (true) {
    while (counter == 0)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* consume the item in next_consumed */
}
```



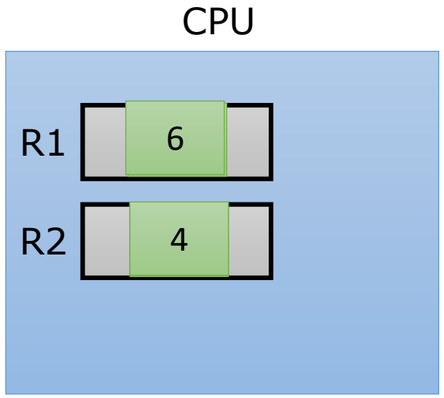
Race Condition

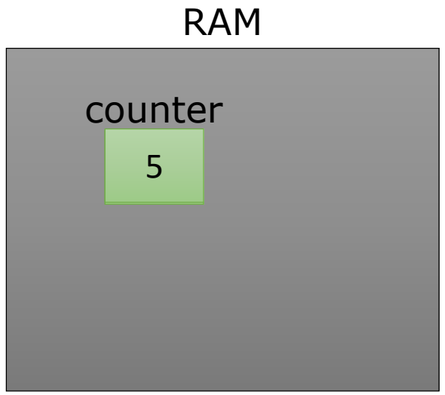
- `counter++` could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- `counter--` could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```





- Consider this execution interleaving with "`count = 5`" initially:
  - S0: producer execute `register1 = counter` {register1 = 5}
  - S1: producer execute `register1 = register1 + 1` {register1 = 6}
  - S2: consumer execute `register2 = counter` {register2 = 5}
  - S3: consumer execute `register2 = register2 - 1` {register2 = 4}
  - S4: producer execute `counter = register1` {counter = 6 }
  - S5: consumer execute `counter = register2` {counter = 4}

Critical Section Problem

- **Consider system of *n* processes {p_0, p_1, … p_{n-1}}**

- **Each process has critical section segment of code**
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section

- ***Critical section problem* is to design a protocol to solve this**

- **Each process must ask permission to enter critical section in entry section, may follow critical section with exit section, then remainder section**

Critical Section

- **General structure of process p_i is**

```
do {
        entry section
                critical section
        exit section
                remainder section
} while (true);
```

Solution to Critical-Section Problem

1. **Mutual Exclusion - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections**

2. **Progress - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely**

Solution to Critical-Section Problem

3. Bounded Waiting - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

- Assume that each process executes at a nonzero speed
- No assumption concerning **relative speed** of the ***n*** processes

Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
  - Essentially free of race conditions in kernel mode

Synchronization Hardware

- **Many systems provide hardware support for critical section code**
- **All solutions below based on idea of locking**
  - Protecting critical regions via locks
- **Uniprocessors – could disable interrupts**
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable
  - Too risky
- **Modern machines provide special atomic hardware instructions**
  - **Atomic** = non-interruptible
  - Either test memory word and set value
  - Or swap contents of two memory words

Solution to Critical-Section Problem Using `Lock`

```
1  do {
2      acquire lock
3          /*critical section*/
4      release lock
5          /*remainder section*/
6   } while (true);
```

test_and_set Instruction

Definition:

```
1  boolean test_and_set (boolean *target)
2   {
3     boolean rv = *target;
4     *target = true;
5     return rv:
6   }
```

Solution using `test_and_set`

- **Shared boolean variable `lock` , initialized to `false`**

- **Solution:**

```
1  do {
2       while (test_and_set(&lock))
3          ; /* do nothing */
4       /*critical section*/
5       lock = false;
6       /*remainder section*/
7  } while (true);
```

Bounded-Waiting Mutual Exclusion with `test_and_set`

```
1  do {
2       waiting[i] = true;
3       key = true;
4       while (waiting[i] && key)
5          key = test_and_set(&lock);
6       waiting[i] = false;
7       /*critical section*/
8       j = (i + 1) % n;
9       while ((j != i) && !waiting[j])
10         j = (j + 1) % n;
11      if (j == i)
12         lock = false;
13      else
14         waiting[j] = false;
15      /*remainder section*/
16    } while (true);
```

test_and_set on i386 processors

Using Bit Test-and-Set

```
int test_and_set(unsigned char *b) {
   int result;
   asm ("bts $1, %1; sbbl %0, %0":"=r" (result):"m" (*b):"memory");
   return result;
}
```

operand-constraint

(C expression)

assembly instructions

output

input

clobbered

Using Exchange

```
int test_and_set(unsigned char *addr) {
   unsigned char result = 1;
   asm ("xchgb %1, %0":"=m" (*addr),"=r" (result) :"1" (result) :"memory");
   return result;
}
```

BTS: Bit test and set
SBB: Integer subtraction with borrow
XCHG: Exchange reg/mem with register

Source:
http://www.acm.uiuc.edu/sigops/roll_your_own/i386/atomic.html

swap Instruction

Definition:

```
1  void swap (boolean *a, boolean *b)
2   {
3     boolean temp = *a;
4     *a = *b;
5     *b = temp:
6   }
```

Solution using `swap`

- **Shared boolean variable lock initialized to `false`;**
  Each process has a local boolean variable key
- **Solution:**

```
1  do {
2       key = true;
3       while ( key == true)
4             swap (&lock, &key );
5       /*critical section*/
6       lock = false;
7       /*remainder section*/
8
9     } while (true);
```

Implementing Mutex using swap

ARM Semaphore Instruction Mutex Example (ARM Arch. Ref. Manual)

```
on entry    R0          Holds the address of semaphore
            R1          Holds the Process ID requesting the lock

            mvn         R2, #0      ; load the looking value of -1 in R2

spinin      swp         R3, R2, [R0]; m[R0] ->R3, m[R0]<-R2
            cmn         R3,#1       ; anyone else tying to "look"?

            .... OS call to sleep process   ; remember the SKIP when not equal

            beq         spinin      ; yes, wait for my turn

            cmp         R3, #0      ; Is lock free
            strne       R3, [R0]    ; no, restore the lock
            .... OS call to sleep process   ; remember the SKIP when not equal
            bne         spinin

            str         R1,[R0]     ; assume lock ownership with own ID

            ... insert your critical here

spinout     swp         R3, R2, [R0]; look at the lock
            cmn         R3, #1      ; anyone is looking?
            .... OS call to sleep process   ; remember the SKIP when not equal
            beq         spinout
```

Semaphore

- **Synchronization tool that does not require busy waiting**
- **Semaphore `S` – integer variable**
- **Two standard operations modify `S`: `wait()` and `signal()`**
  - Originally called `P()` and `V()`
- **Less complicated**
- **Can only be accessed via two indivisible (atomic) operations**

```
1  wait (S) {
2      while (S <= 0)
3        ; // do nothing
4      S--;
5    }
```

```
1  signal (S) {
2      S++;
3    }
4
5
```

Semaphore as General Synchronization Tool

- **Counting semaphore – integer value can range over an unrestricted domain**
- **Binary semaphore – integer value can range only between 0 and 1; can be simpler to implement**
  - Also known as mutex locks
- **Can implement a counting semaphore `S` as a binary semaphore**
- **Provides mutual exclusion**

```
1  Semaphore mutex; //  initialized to 1
2  do {
3    wait (mutex);
4    /*critical Section*/
5    signal (mutex);
6    /*remainder section*/
7     } while (true);
```

Semaphore Implementation

- **Must guarantee that no two processes can execute `wait()` and `signal()` on the same semaphore at the same time**
- **Thus, implementation becomes the critical section problem where the wait and signal code are placed in the crtical section.**
  - Could now have busy waiting in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied
- **Note that applications may spend lots of time in critical sections and therefore this is not a good solution.**

Implementing Semaphores

- **Since semaphores is actually a critical section problem, it may be implemented by various approaches.**
- **For example, `test_and_set` may be used to implement a binary semaphore as following**

```
1  void wait(S) {
2        while(test_and_set(&S));
3  }
4  void signal(S) {
5        S = 0;
6  }
```

Are we done yet?

- **Not quite!**
- **The shared variable solutions require busy waiting**
  - bad: this can be inefficient - the locked out thread needs to be running
  - worse: can deadlock if the waiting process has higher priority in the scheduler
- **We'd usually prefer a blocking solution**

Semaphore Implementation with no Busy waiting

- **With each semaphore there is an associated waiting queue. Each entry in a waiting queue has two data items:**
  - value (of type integer)
  - pointer to next record in the list

- **Two operations:**
  - **`block`** – place the process invoking the operation on the appropriate waiting queue.
  - **`wakeup`** – remove one of processes in the waiting queue and place it in the ready queue.

Semaphore Implementation with no Busy waiting (Cont.)

- **Implementation of wait:**

```
1  wait(semaphore *S) {
2        S->value--;
3        if (S->value < 0) {
4              add this process to S->list;
5              block();
6        }
7    }
```

- **Implementation of signal:**

```
1  signal(semaphore *S) {
2        S->value++;
3        if (S->value <= 0)   {
4              remove a process P from S->list;
5              wakeup(P);
6        }
7    }
```

Semaphore Implementation with no Busy waiting (Cont.)

- **No two process can execute `wait()` and `signal()` on the same semaphore at the same time – a critical section problem (semaphore is critical section).**
  - Can solve this by simply disabling interrupt on uniprocessors.
- **Note that this implementation does not completely eliminate busy waiting.**
  - But remove busy waiting from the entire section, i.e., entire `wait()` or `signal()`, to the critical sections of `wait()` or `signal()`, i.e., semaphore itself.
  - Thus, the critical section is almost never occupied, and busy waiting occurs rarely, and then for only a short time

Deadlock and Starvation

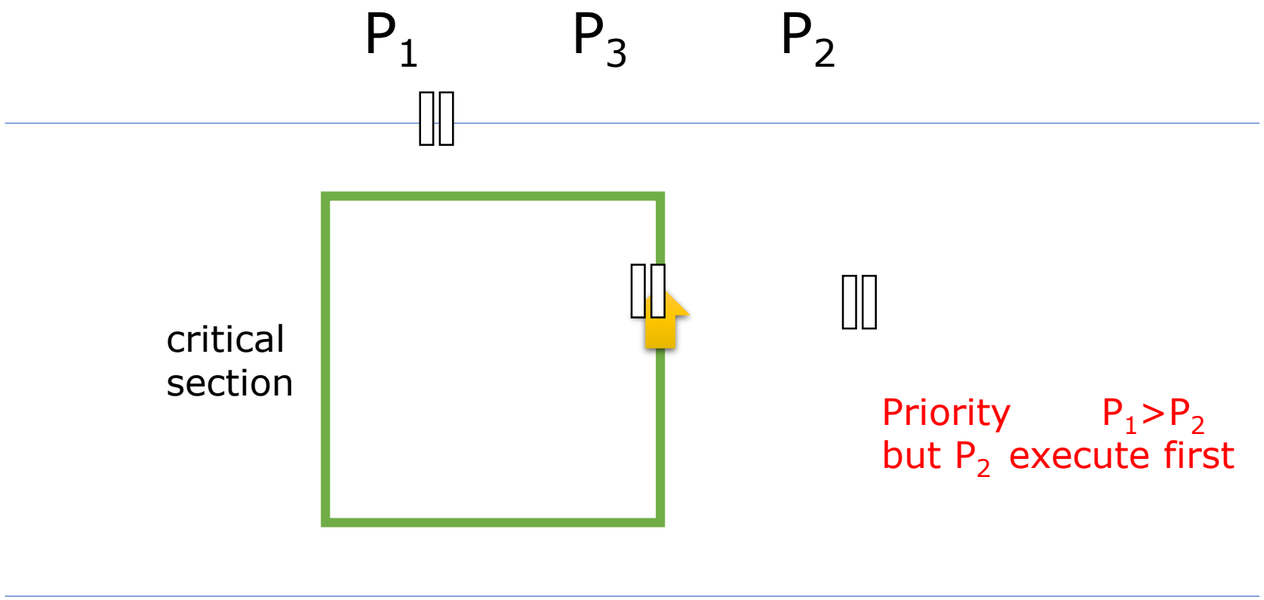
- **Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes**
- **Let S and Q be two semaphores initialized to 1**

```
        P0                          P1
wait(S);                    wait(Q);
wait(Q);                    wait(S);
  .                           .
  .                           .
  .                           .
signal(S);                  signal(Q);
signal(Q);                  signal(S);
```

- **Starvation – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended**
- **Priority Inversion - Scheduling problem when lower-priority process holds a lock needed by higher-priority process**

Priority Inversion

- **Priority $P_1 > P_2 > P_3$**
- **Arrival order P_3, P_2, P_1**



Solution:
Temporarily increase P_3 priority to P_1

Classical Problems of Synchronization

- **Classical problems used to test newly-proposed synchronization schemes**
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem

Bounded-Buffer Problem

- **`n` buffers, each can hold one item**
- **Semaphore `mutex` initialized to the value 1**
- **Semaphore `full` initialized to the value 0**
- **Semaphore `empty` initialized to the value `n`**

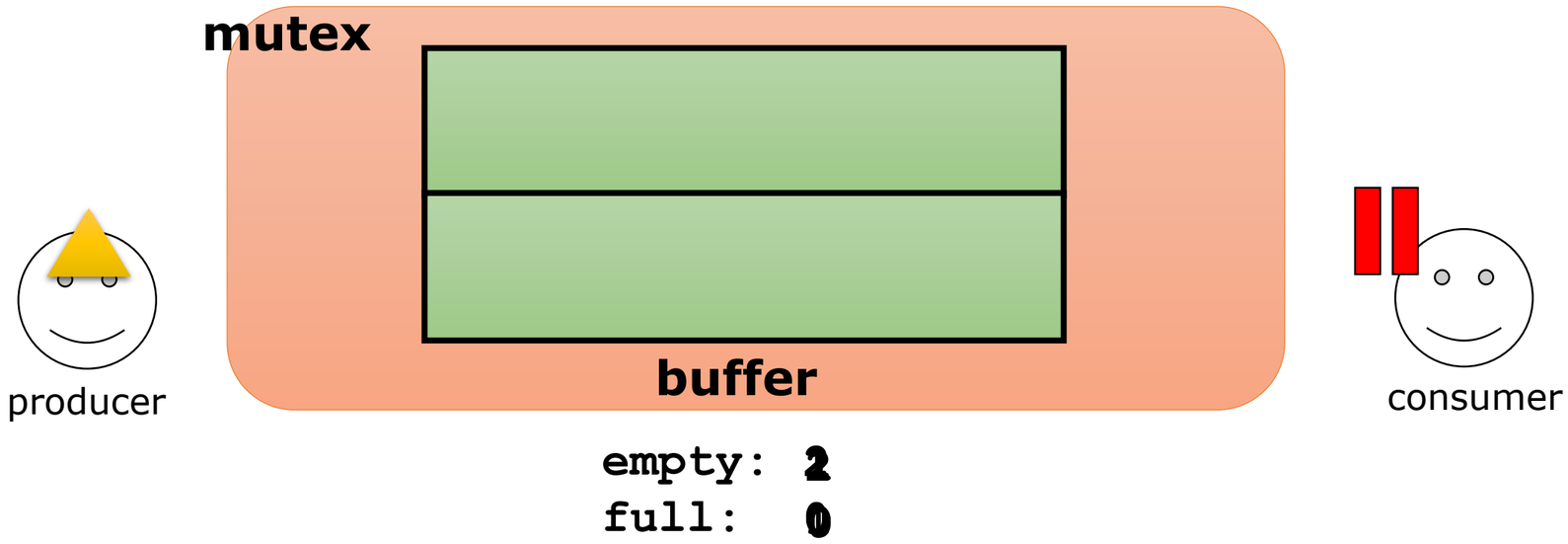
Bounded Buffer Problem (Cont.) -1

Producer

```
1  do {
2      ...
3      /* produce an item */
4      ...
5      wait(empty);
6      wait(mutex);
7
8      /* add the item */
9
10     signal(mutex);
11     signal(full);
12 } while (true);
```

Consumer

```
1  do {
2      wait(full);
3      wait(mutex);
4        ...
5        /* remove an item */
6        ...
7      signal(mutex);
8      signal(empty);
9        ...
10       /* consume the item */
11       ...
12 } while (true);
```



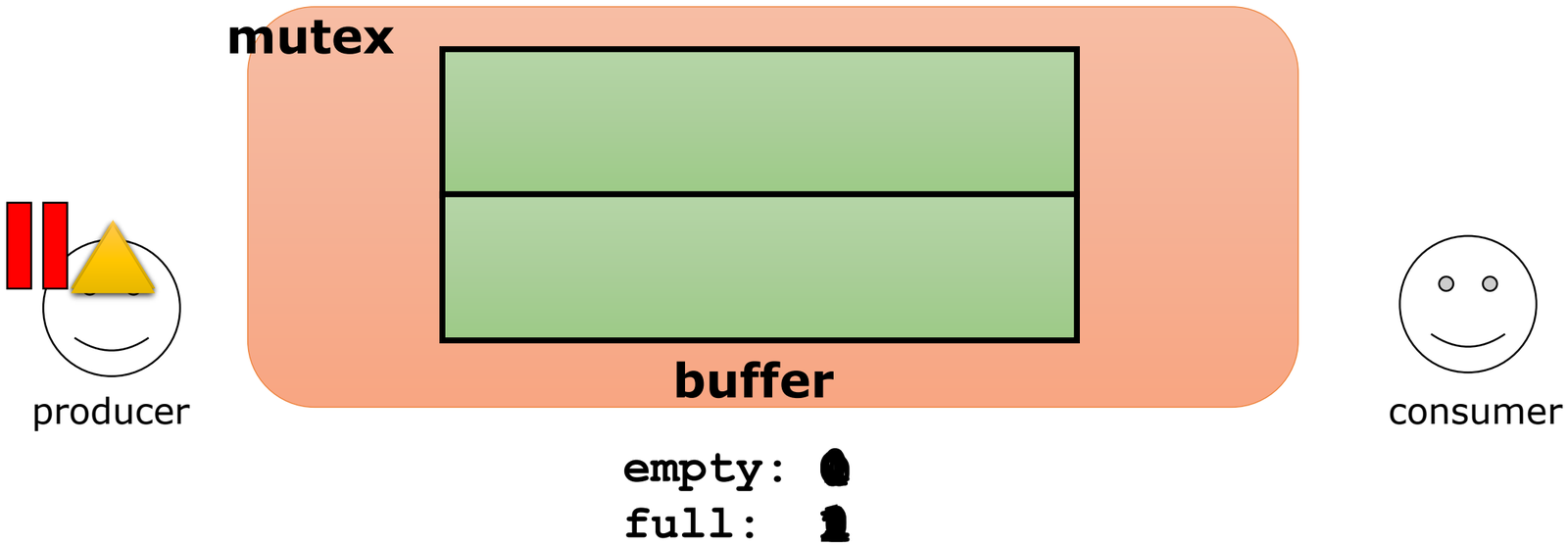
Bounded Buffer Problem (Cont.) -2

Producer

```
1  do {
2      ...
3      /* produce an item */
4      ...
5      wait(empty);
6      wait(mutex);
7
8      /* add the item */
9
10     signal(mutex);
11     signal(full);
12 } while (true);
```

Consumer

```
1  do {
2      wait(full);
3      wait(mutex);
4        ...
5        /* remove an item */
6        ...
7      signal(mutex);
8      signal(empty);
9        ...
10       /* consume the item */
11       ...
12 } while (true);
```



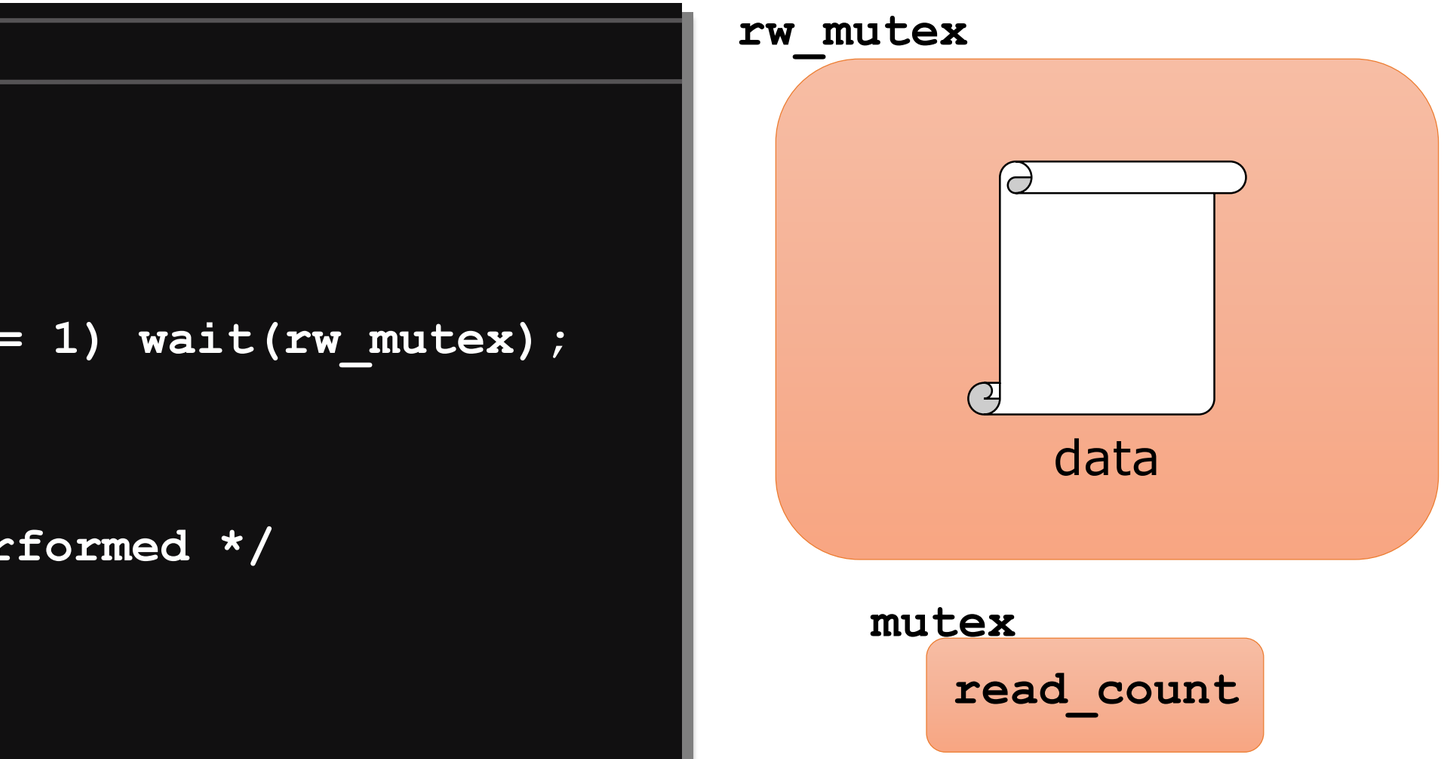
Readers-Writers Problem

- **A data set is shared among a number of concurrent processes**
  - Readers – only read the data set; they do ***not*** perform any updates
  - Writers – can both read and write
- **Problem – allow multiple readers to read at the same time**
  - Only one single writer can access the shared data at the same time
- **Several variations of how readers and writers are treated – all involve priorities**

Shared Data

- Shared Data
  - Structure of a reader process

```
1  do {
2       wait(rw_mutex);
3       read_count++;
4       if (read_count == 1) wait(rw_mutex);
5       signal(mutex);
6       ...
7       /* reading is performed */
8       ...
9       wait(mutex);
10      read_count--;
11      if (read_count == 0) signal(rw_mutex);
12      signal(mutex);
13   } while (true);
```



Readers-Writers Problem Variations

- ***First*** **variation – no reader kept waiting unless writer has permission to use shared object**
- ***Second*** **variation – once writer is ready, it performs write ASAP**
- **Both may have starvation leading to even more variations**
- **Problem is solved on some systems by kernel providing reader-writer locks**

Dining-Philosophers Problem

- **Philosophers spend their lives thinking and eating**
- **Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl**
  - Need both to eat, then release both when done
- **In the case of 5 philosophers**
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem Algorithm

The structure of Philosopher *i*:

```
do  {
        wait ( chopstick[i] );
        wait ( chopStick[(i + 1) % 5] );

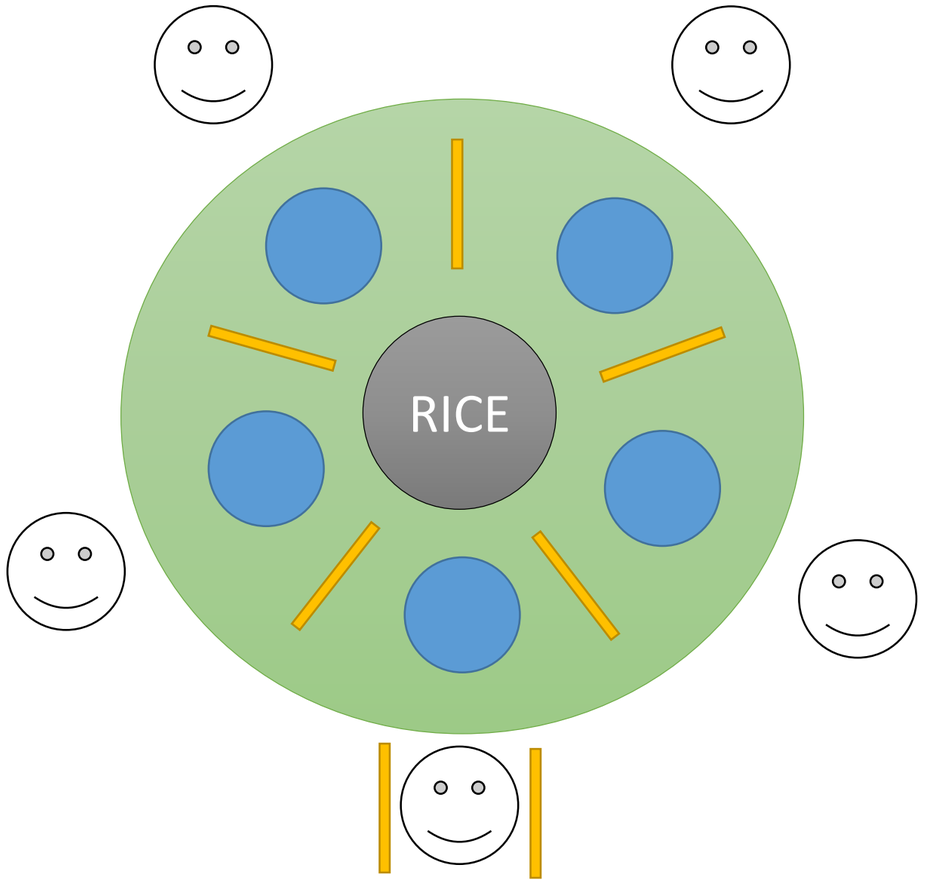
            //  eat

        signal ( chopstick[i] );
        signal ( chopstick[(i + 1) % 5] );

            //  think

} while (TRUE);
```

What is the problem with this algorithm?



Dining-Philosophers Problem Algorithm

The structure of Philosopher *i*:

```
do  {
        wait ( chopstick[i] );
        wait ( chopStick[(i + 1) % 5] );

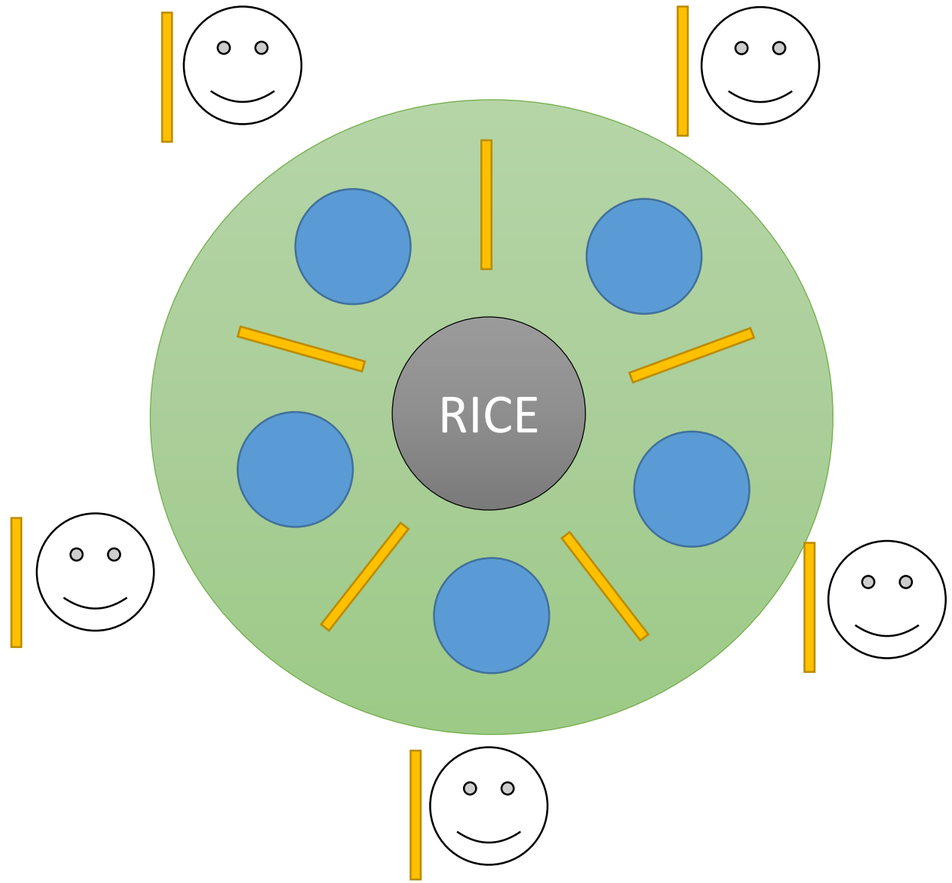
            //  eat

        signal ( chopstick[i] );
        signal ( chopstick[(i + 1) % 5] );

            //  think

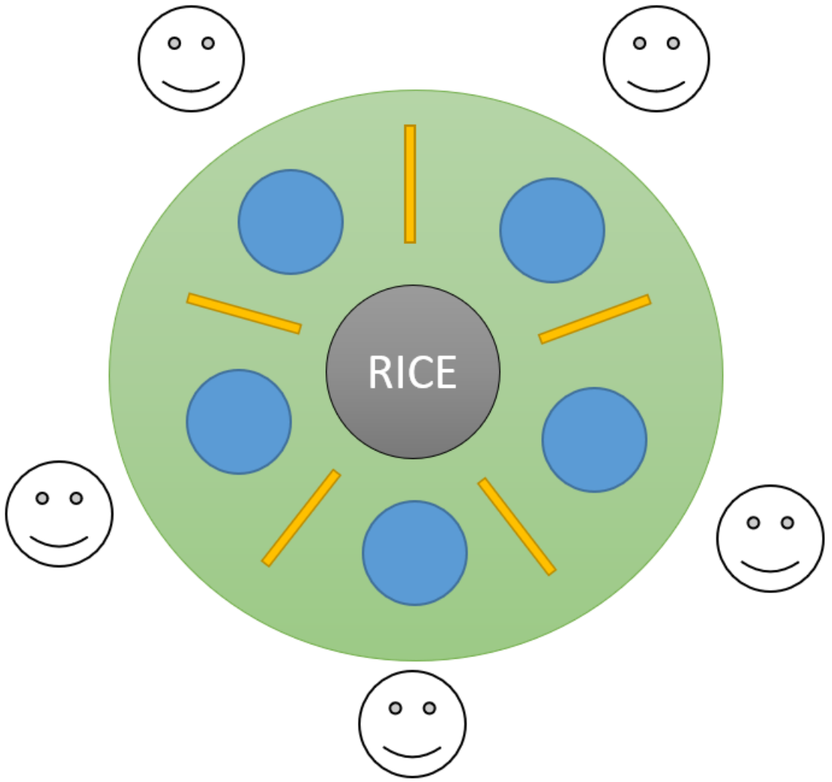
} while (TRUE);
```

What is the problem with this algorithm?



Dining-Philosophers Problem (Cont.)

- **Deadlock may occur**
- **Possible remedies**
  - Allow at most four philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
    - She must pick them up in a critical section.
  - Use an asymmetric solution
    - Odd philosopher picks up first her left chopstick and then her right chopstick
    - Even philosopher pick up her right chopstick and then her left chopstick.



Problems with Semaphores

- **Still too low-level**
- **Semaphore operations are scattered through code**
- **`wait()` and `signal()` are used in pairs: how do you know which pairs match?**
  - Require correct use of semaphore operations:
    - **`signal (mutex)`** … **`wait (mutex)`**
    - **`wait (mutex)`** … **`wait (mutex)`**
    - Omitting of **`wait (mutex)`** or **`signal (mutex)`** (or both)
  - These errors may be discovered only if several processes are simultaneously active in their critical sections.
    - This situation may not always be reproducible – hard to debug

Monitors

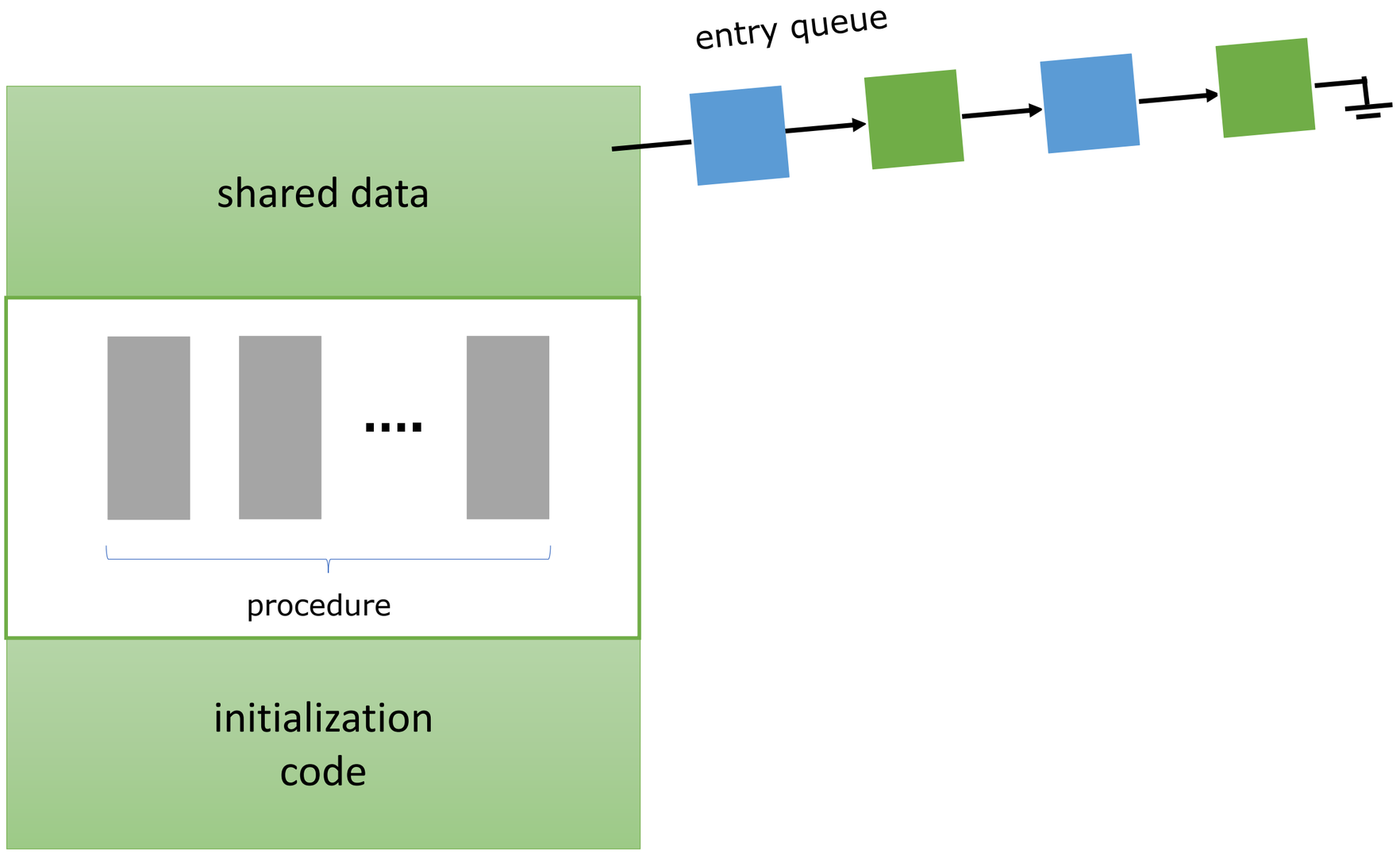
- **A high-level abstraction that provides a convenient and effective mechanism for process synchronization**
- ***Abstract data type*, internal variables only accessible by code within the procedure**
- **Only one process may be active within the monitor at a time**
- **But not powerful enough to model some synchronization schemes**

```
monitor monitor-name
{
      // shared variable declarations
      procedure F1 (…) {……}

      procedure Fn (…) {……}

     initialization code (…) {……..}
}
```

Schematic view of a Monitor

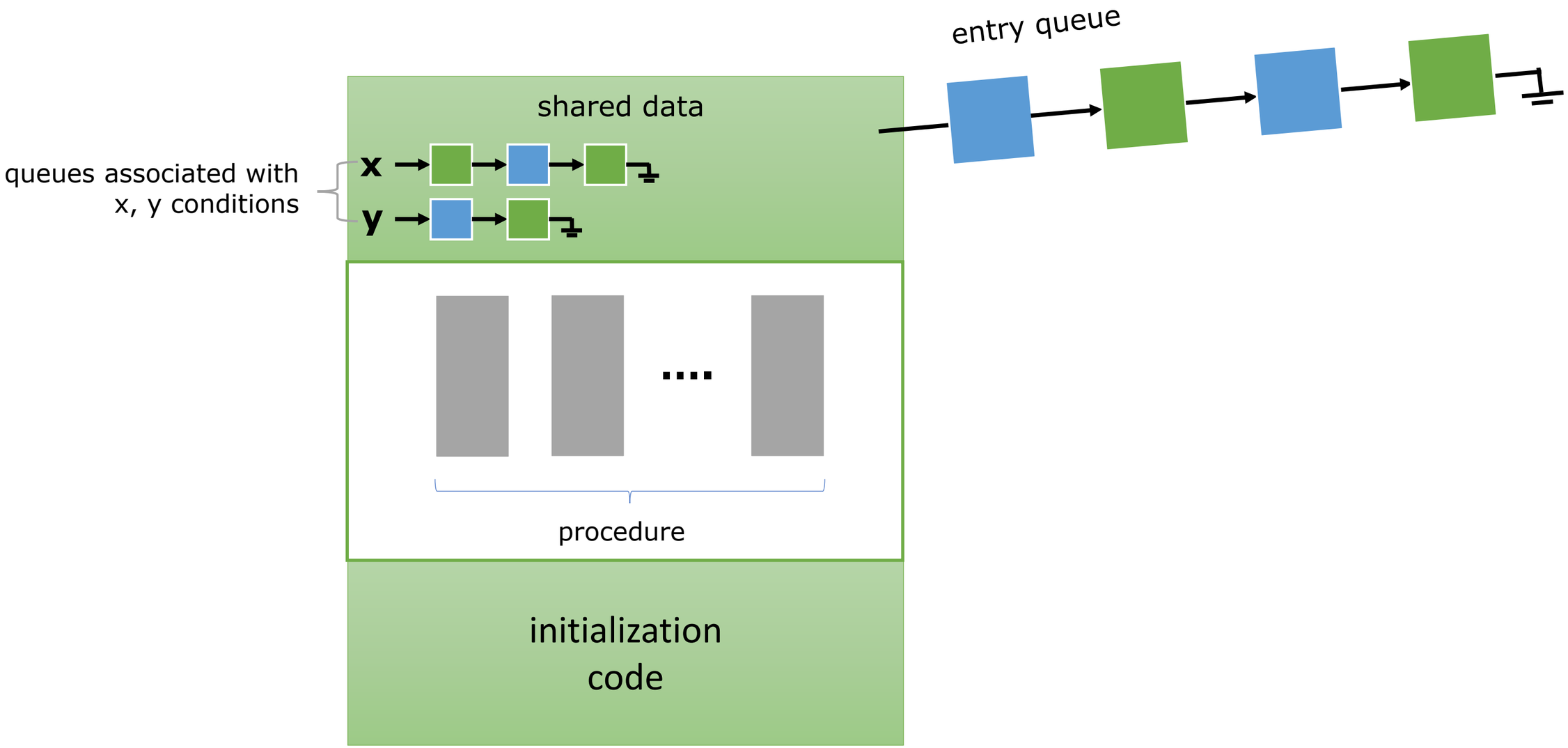


Condition Variables

- `condition x, y;`

- **Two operations on a condition variable:**
  - `x.wait()` – a process that invokes the operation is suspended until `x.signal()`
  - `x.signal()` – resumes one of processes (if any) that invoked `x.wait()`
    - If no `x.wait()` on the variable, then it has no effect on the variable

Monitor with Condition Variables

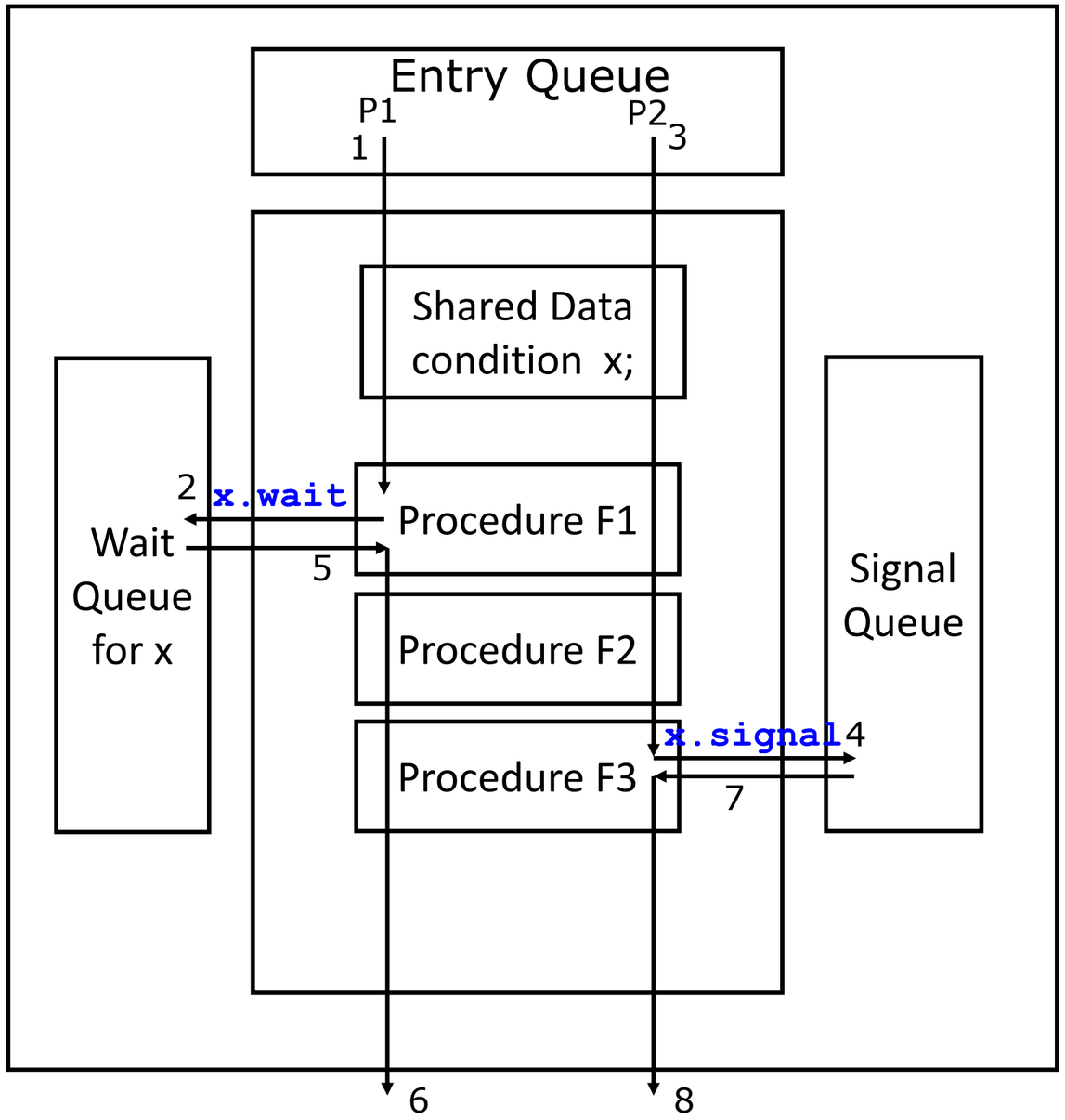


Condition Variables Choices

- **If process P invokes `x.signal()`, with Q in `x.wait()` state, what should happen next?**
  - If Q is resumed, then P must wait

- **Options include**
  - **Signal and wait** – P waits until Q leaves monitor or waits for another condition
  - **Signal and continue** – Q waits until P leaves the monitor or waits for another condition

Hoare Semantics



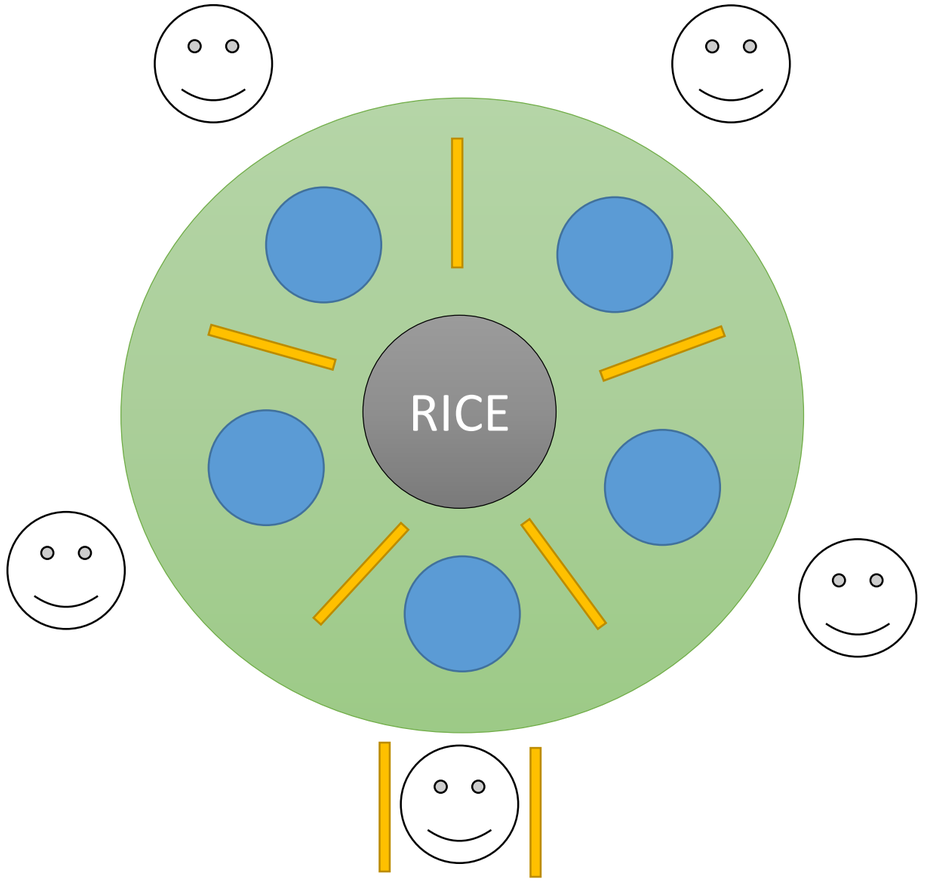
1. P1 enters the monitor
2. P1 waits for a resource
3. P2 enters the monitor
4. P2 signals on the resource and enters the signal queue
5. P1 re-enters the monitor
6. P1 leaves the monitor
7. P2 re-enters the monitor
8. P2 leaves the monitor
9. Another process can be admitted from the entry queue

Solution to Dining Philosophers

```
monitor DiningPhilosophers
    {
        enum { THINKING, HUNGRY, EATING) state [5] ;
        condition self [5];

        void test (int i) {
                if ( (state[(i + 4) % 5] != EATING) &&
                     (state[i] == HUNGRY) &&
                     (state[(i + 1) % 5] != EATING) ) {
                          state[i] = EATING ;
                           self[i].signal() ;
                }
         }

        void pickup (int i) {
               state[i] = HUNGRY;
               test(i);
               if (state[i] != EATING) self [i].wait;
        }
```

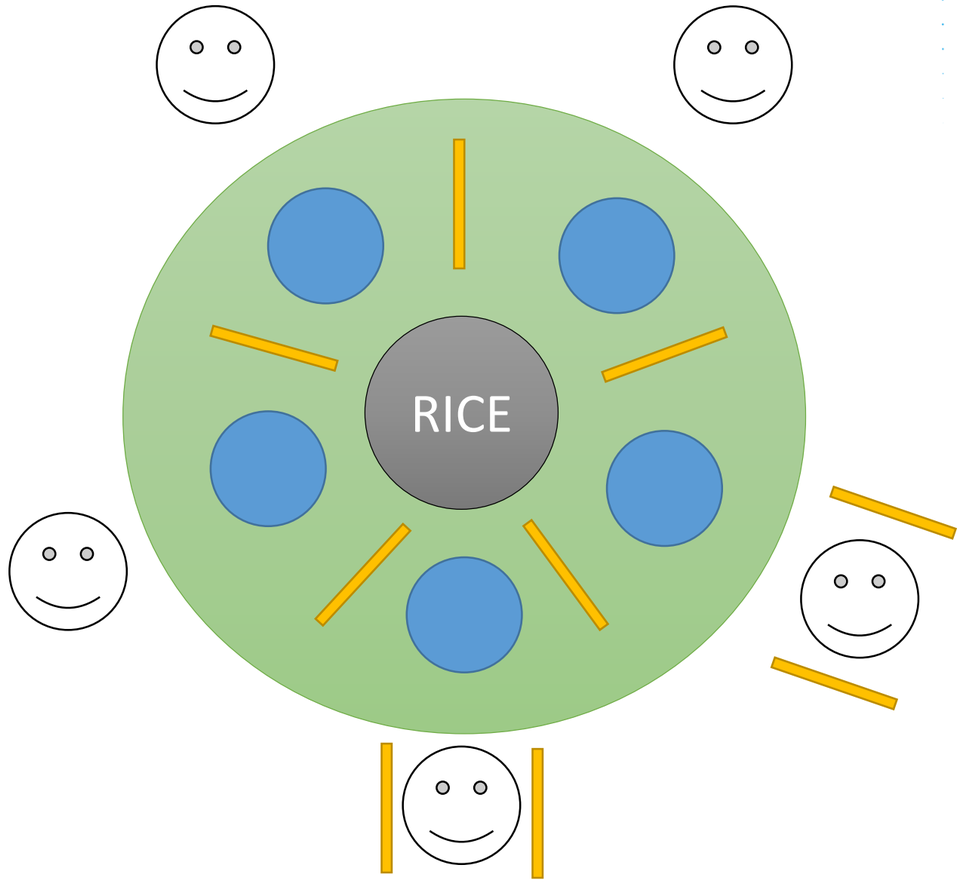


Solution to Dining Philosophers (Cont.)

```
        void putdown (int i) {
                state[i] = THINKING;
                        // test left and right neighbors
                 test((i + 4) % 5);
                 test((i + 1) % 5);
            }

          initialization_code() {
                for (int i = 0; i < 5; i++)
                state[i] = THINKING;
        }
}
```

```
void test (int i) {
    if ( (state[(i + 4) % 5] != EATING) &&
         (state[i] == HUNGRY) &&
         (state[(i + 1) % 5] != EATING) ) {
             state[i] = EATING ;
             self[i].signal() ;
     }
}
```



Solution to Dining Philosophers (Cont.)

- Each philosopher *i* invokes the operations `pickup()` and `putdown()` in the following sequence:

```
DiningPhilosophers.pickup(i);
        /* EAT */
DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible

Monitor Implementation Using Semaphores

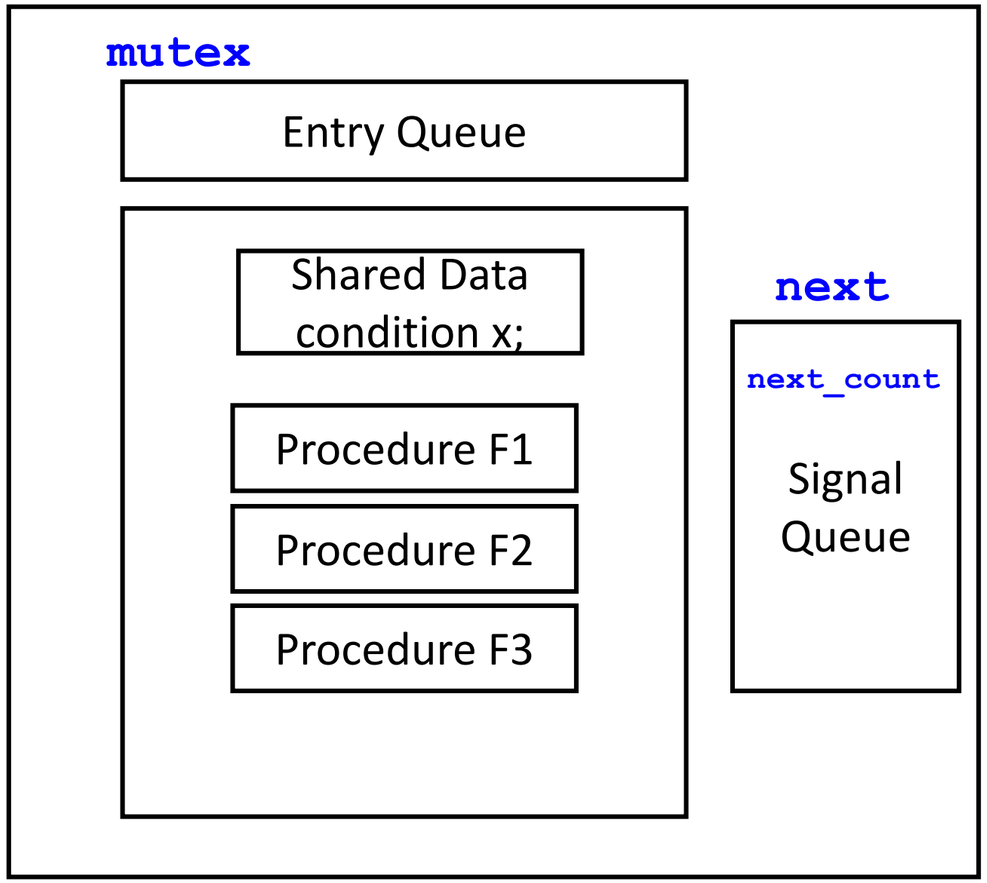
- **Variables**

```
semaphore mutex;  // (initially  = 1)
semaphore next;     // (initially  = 0)
int next_count = 0;
```

- **Each procedure F_i will be replaced by**

```
wait(mutex);
    …
/* body of F */
    …
if (next_count > 0)
  signal(next)
else
  signal(mutex);
```

- **Mutual exclusion within a monitor is ensured**



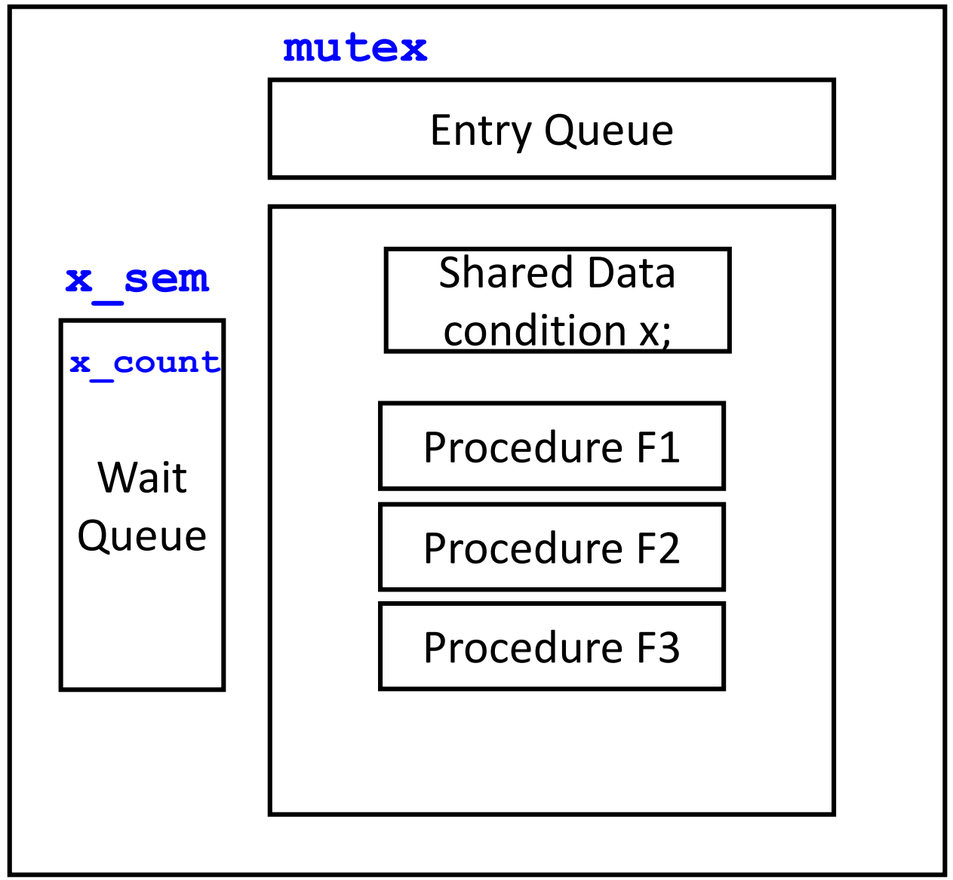
Monitor Implementation – Condition Variables

- For each condition variable **x**, we have:

```
semaphore x_sem; // (initially  = 0)
int x_count = 0;
```

- The operation **x.wait** can be implemented as:

```
x-count++;
if (next_count > 0)
  signal(next);
else
  signal(mutex);
wait(x_sem);
x-count--;
```



Monitor Implementation (Cont.)

- **The operation `x.signal` can be implemented as:**

```
if (x-count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```

Resuming Processes within a Monitor

- **If several processes queued on condition `x`, and `x.signal()` executed, which should be resumed?**
- **FCFS frequently not adequate**
- **conditional-wait construct of the form `x.wait(c)`**
  - Where c is **priority number**
  - Process with lowest number (highest priority) is scheduled next

Semaphore Example Code

- Nuclear Reactor System
  - MicroC/OS RTOS
- Reentrancy and Semaphore
  - Nucleus RTOS
- Task Synchronization
  - eCos RTOS
  - Linux

The Nuclear Reactor System

- MicroC/OS RTOS
- Semaphore-related functions
  - OSSemPost(): release the semaphore
  - OSSemPend(): take the semaphore
  - OSSemCreate(): initialize the semaphore
- Related data structures
  - OS_EVENT: the data representing the semaphore
  - WAIT_FOREVER: indicates that the task making the call is willing to wait forever
- Other functions
  - OSTimeDly(): block functions

The Code

- The data structures

```
#define TASK_PRIORITY_READ 11
#define TASK_PRIORITY_CONTROL 12
#define STK_SIZE 1024
static unsigned int ReadStk [STK_SIZE];
static unsigned int ControlStk [STK_SIZE];

static int iTemperatures[2];
OS_EVENT *p_semTemp;
```

The Code

- The main function

```
void main (void)
{
   /* Initialize (but do not start) the RTOS */
   OSInit ();

   /* Tell the RTOS about our tasks */
   OSTaskCreate (vReadTemperatureTask,  NULLP,
      (void *)&ReadStk[STK_SIZE], TASK_PRIORITY_READ);
   OSTaskCreate (vControlTask,  NULLP,
      (void *)&ControlStk[STK_SIZE], TASK_PRIORITY_CONTROL);

   /* Start the RTOS.  (This function never returns.) */
   OSStart ();
}
```

The Code

- Two tasks

```
void vReadTemperatureTask (void)
{
   while (TRUE)
   {
      OSTimeDly (5); /* Delay about 1/4 second */

      OSSemPend (p_semTemp, WAIT_FOREVER);
      !! read in iTemperatures[0];
      !! read in iTemperatures[1];
      OSSemPost (p_semTemp);
   }
}
```

```
void vControlTask (void)
{
   p_semTemp = OSSemInit (1);
   while (TRUE)
   {
      OSSemPend (p_semTemp, WAIT_FOREVER);
      if (iTemperatures[0] != iTemperatures[1])
         !! Set off howling alarm;
      OSSemPost (p_semTemp);

      !! Do other useful work
   }
}
```

A potential bug!

Initializing Semaphores in Nuclear Reactor

- How do you know that OSSemCreate happens before OSSemPend in vReadTemperatureTask?
  - Because of delay by calling OSTimeDly(5)?
    - Some higher priority task might take up all the delay introduced!
  - Change Task Priorities?
    - Someone later might change back the task priorities and not know of the time bomb!
- Correct solution
  - Place OSSemCreate BEFORE OSStart in main!

Reentrancy and Semaphores

- Now adding a semaphore to the previous code (using Nucleus RTOS system calls)

```
static int cErrors;

void vCountErrors (int cNewErrors)
{
   cErrors += cNewErrors;
}
```

```
static int cErrors;
static NU_SEMAPHORE semErrors;

void vCountErrors (int cNewErrors)
{
   NU_Obtain_Semaphore (&semErrors, NU_SUSPEND);
   cErrors += cNewErrors;
   NU_Release_Semaphore (&semErrors);
}
```

Atomic now!

Mutex Task Synchronization in eCos

- Two tasks share a variable **`gSharedVariable`**
  - **`incrementTask`**: increments the variable value
  - **`decrementTask`**: decrements the variable value
- A mutex is used to protect this shared variable

Mutex Task Synchronization in eCos (main program)

```
#include <cyg/kernel/kapi.h>
#include <cyg/infra/diag.h>
cyg_mutex_t sharedVariableMutex;
int32_t gSharedVariable = 0;


void cyg_user_start(void) {
  /* Create the mutex for accessing the shared variable. */
  cyg_mutex_init(&sharedVariableMutex);


  /* Create the increment and decrement tasks. */
  cyg_thread_create(INCREMENT_TASK_PRIORITY, incrementTask,
        (cyg_addrword_t)0, "Increment Task",
                (void *)incrementTaskStack, INCREMENT_TASK_STACK_SIZE,
                &incrementTaskHdl, &incrementTaskObj);
  cyg_thread_create(DECREMENT_TASK_PRIORITY, decrementTask,
                (cyg_addrword_t)0, "Decrement Task",
                (void *)decrementTaskStack, DECREMENT_TASK_STACK_SIZE,
                &decrementTaskHdl, &decrementTaskObj);


  /* Notify the scheduler to start running the tasks. */
  cyg_thread_resume(incrementTaskHdl);
  cyg_thread_resume(decrementTaskHdl);
  diag_printf("eCos mutex example.\\n");
}
```

Mutex Task Synchronization in eCos (incrementTask)

```
void incrementTask(cyg_addrword_t data) {
 while (1) {
    /* Delay for 3 seconds. */
    cyg_thread_delay(TICKS_PER_SECOND * 3);

    /* Wait for the mutex to become available. */
    cyg_mutex_lock(&sharedVariableMutex);
    gSharedVariable++;
    diag_printf("Increment Task: shared variable
    value is %d\\n", gSharedVariable);

    /* Release the mutex. */
    cyg_mutex_unlock(&sharedVariableMutex);
 }
}
```

Mutex Task Synchronization in eCos (decrementTask)

```
void decrementTask(cyg_addrword_t data) {
 while (1) {
    /* Delay for 7 seconds. */
    cyg_thread_delay(TICKS_PER_SECOND * 7);

    /* Wait for the mutex to become available. */
    cyg_mutex_lock(&sharedVariableMutex);
    gSharedVariable--;
    diag_printf("Decrement Task: shared variable
    value is %d\\n", gSharedVariable);

    /* Release the mutex. */
    cyg_mutex_unlock(&sharedVariableMutex);
 }
}
```

Mutex Task Synchronization in Linux (main program)

```
#include <pthread.h>
pthread_mutex_t sharedVariableMutex;
int32_t gSharedVariable = 0;

int main(void) {
  /* Create the mutex for accessing the shared variable using
     * the default attributes. */
  pthread_mutex_init(&sharedVariableMutex, NULL);

  /* Create the increment and decrement tasks */
  pthread_create(&incrementTaskObj, NULL,
                 (void *)incrementTask, NULL);
  pthread_create(&decrementTaskObj, NULL,
                 (void *)decrementTask, NULL);

  /* Allow the tasks to run. */ pthread_join(incrementTaskObj, NULL);
  pthread_join(decrementTaskObj, NULL);
  return 0;
}
```

Mutex Task Synchronization in Linux (incrementTask)

```
#include <stdio.h>
#include <unistd.h>

void incrementTask(void *param) {
  while (1) {
       /* Delay for 3 seconds. */
       sleep(3);

       /* Wait for the mutex before accessing the GPIO
       registers. */
       pthread_mutex_lock(&sharedVariableMutex);
       gSharedVariable++;
       printf("Increment Task: shared variable value is %d\\n",
             gSharedVariable);
       /* Release the mutex for other task to use. */
       pthread_mutex_unlock(&sharedVariableMutex);
  }
}
```

Mutex Task Sychronization in Linux (decrementTask)

```
void decrementTask(void *param) {
 while (1) {
    /* Delay for 7 seconds. */
    sleep(7);

    /* Wait for the mutex to become available. */
    pthread_mutex_lock(&sharedVariableMutex);
    gSharedVariable--;
    printf("Decrement Task: shared variable value
 is     %d\\n", gSharedVariable);

    /* Release the mutex. */
    pthread_mutex_unlock(&sharedVariableMutex);
 }
}
```



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 26.7: SimpleArray.java
2  // Class that manages an integer array to be shared by multiple threads.
3  import java.util.Arrays;
4  import java.util.Random;
5
6  public class SimpleArray // CAUTION: NOT THREAD SAFE!
7  {
8     private final int[] array; // the shared integer array
9     private int writeIndex = 0; // index of next element to be written
10    private final static Random generator = new Random();
11
12    // construct a SimpleArray of a given size
13    public SimpleArray( int size )
14    {
15       array = new int[ size ];
16    } // end constructor
17
18    // add a value to the shared array
19    public void add( int value )
20    {
21       int position = writeIndex; // store the write index
22
```

Fig. 26.7 | Class that manages an integer array to be shared by multiple threads. (Part 1 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
23          try
24          {
25             // put thread to sleep for 0-499 milliseconds
26             Thread.sleep( generator.nextInt( 500 ) );
27          } // end try
28          catch ( InterruptedException ex )
29          {
30             ex.printStackTrace();
31          } // end catch
32
33          // put value in the appropriate element
34          array[ position ] = value;
35          System.out.printf( "%s wrote %2d to element %d.\n",
36             Thread.currentThread().getName(), value, position );
37
38          ++writeIndex; // increment index of element to be written next
39          System.out.printf( "Next write index: %d\n", writeIndex );
40       } // end method add
41
```

Fig. 26.7 | Class that manages an integer array to be shared by multiple threads. (Part 2 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
42      // used for outputting the contents of the shared integer array
43      public String toString()
44      {
45         return "\nContents of SimpleArray:\n" + Arrays.toString( array );
46      } // end method toString
47   } // end class SimpleArray
```

Fig. 26.7 | Class that manages an integer array to be shared by multiple threads. (Part 3 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 26.8: ArrayWriter.java
2  // Adds integers to an array shared with other Runnables
3  import java.lang.Runnable;
4
5  public class ArrayWriter implements Runnable
6  {
7     private final SimpleArray sharedSimpleArray;
8     private final int startValue;
9
10    public ArrayWriter( int value, SimpleArray array )
11    {
12       startValue = value;
13       sharedSimpleArray = array;
14    } // end constructor
15
16    public void run()
17    {
18       for ( int i = startValue; i < startValue + 3; i++ )
19       {
20          sharedSimpleArray.add( i ); // add an element to the shared array
21       } // end for
22    } // end method run
23 } // end class ArrayWriter
```

Fig. 26.8 | Adds integers to an array shared with other `Runnables`.

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.

```
1  // Fig 26.9: SharedArrayTest.java
2  // Executes two Runnables to add elements to a shared SimpleArray.
3  import java.util.concurrent.Executors;
4  import java.util.concurrent.ExecutorService;
5  import java.util.concurrent.TimeUnit;
6
7  public class SharedArrayTest
8  {
9     public static void main( String[] arg )
10    {
11       // construct the shared object
12       SimpleArray sharedSimpleArray = new SimpleArray( 6 );
13
14       // create two tasks to write to the shared SimpleArray
15       ArrayWriter writer1 = new ArrayWriter( 1, sharedSimpleArray );
16       ArrayWriter writer2 = new ArrayWriter( 11, sharedSimpleArray );
17
18       // execute the tasks with an ExecutorService
19       ExecutorService executor = Executors.newCachedThreadPool();
20       executor.execute( writer1 );
21       executor.execute( writer2 );
22
23       executor.shutdown();
```

Fig. 26.9 | Executes two `Runnables` to insert values in a shared array. (Part 1 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
24
25          try
26          {
27             // wait 1 minute for both writers to finish executing
28             boolean tasksEnded = executor.awaitTermination(
29                1, TimeUnit.MINUTES );
30
31             if ( tasksEnded )
32                System.out.println( sharedSimpleArray ); // print contents
33             else
34                System.out.println(
35                   "Timed out while waiting for tasks to finish." );
36          } // end try
37          catch ( InterruptedException ex )
38          {
39             System.out.println(
40                "Interrupted while waiting for tasks to finish." );
41          } // end catch
42       } // end main
43    } // end class SharedArrayTest
```

Fig. 26.9 | Executes two `Runnable`s to insert values in a shared array. (Part 2 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.

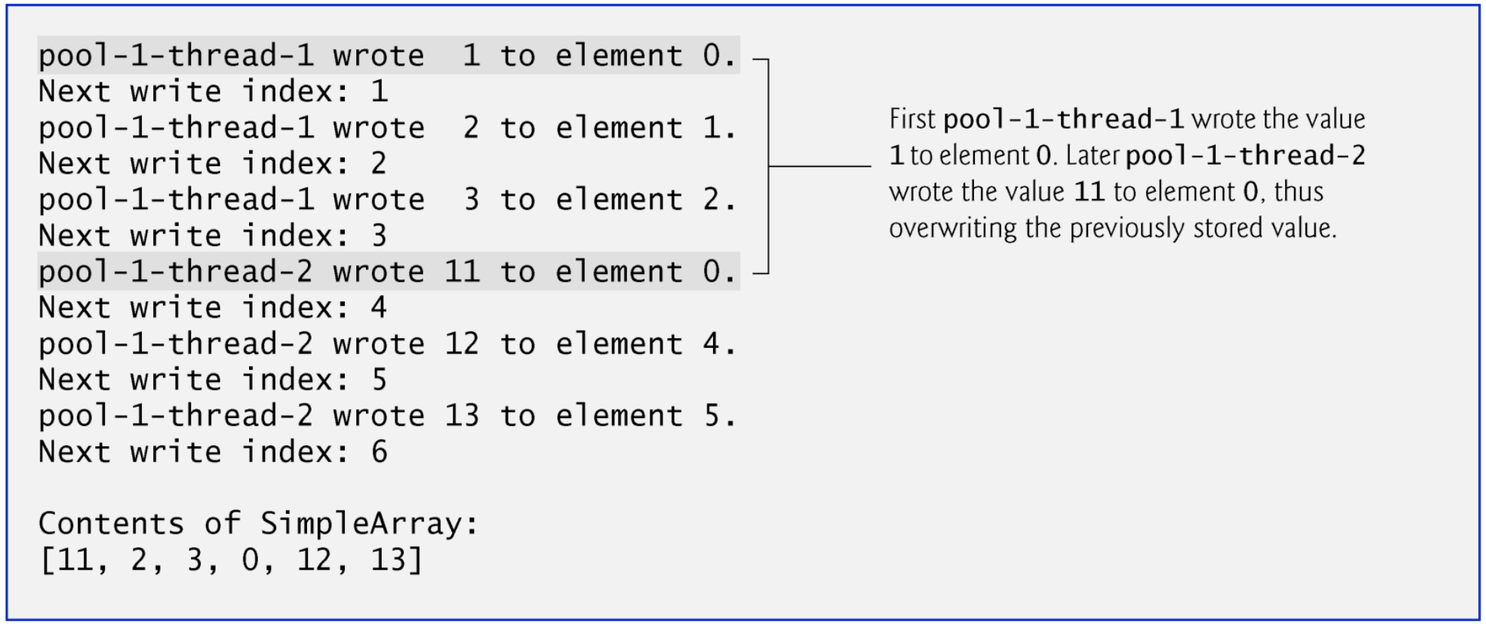


Fig. 26.9 | Executes two `Runnable`s to insert values in a shared array. (Part 3 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 26.10: SimpleArray.java
2  // Class that manages an integer array to be shared by multiple threads
3  // threads with synchronization.
4  import java.util.Arrays;
5  import java.util.Random;
6
7  public class SimpleArray
8  {
9     private final int[] array; // the shared integer array
10    private int writeIndex = 0; // index of next element to be written
11    private final static Random generator = new Random();
12
13    // construct a SimpleArray of a given size
14    public SimpleArray( int size )
15    {
16       array = new int[ size ];
17    } // end constructor
18
```

Fig. 26.10 | Class that manages an integer array to be shared by multiple threads with synchronization. (Part 1 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
19      // add a value to the shared array
20      public synchronized void add( int value )
21      {
22         int position = writeIndex; // store the write index
23
24         try
25         {
26            // put thread to sleep for 0-499 milliseconds
27            Thread.sleep( generator.nextInt( 500 ) );
28         } // end try
29         catch ( InterruptedException ex )
30         {
31            ex.printStackTrace();
32         } // end catch
33
34         // put value in the appropriate element
35         array[ position ] = value;
36         System.out.printf( "%s wrote %2d to element %d.\n",
37            Thread.currentThread().getName(), value, position );
38
39         ++writeIndex; // increment index of element to be written next
40         System.out.printf( "Next write index: %d\n", writeIndex );
41      } // end method add
```

Only one thread at a time can execute this method

Fig. 26.10 | Class that manages an integer array to be shared by multiple threads with synchronization. (Part 2 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
42
43      // used for outputting the contents of the shared integer array
44      public String toString()
45      {
46         return "\nContents of SimpleArray:\n" + Arrays.toString( array );
47      } // end method toString
48   } // end class SimpleArray
```

```
pool-1-thread-1 wrote  1 to element 0.
Next write index: 1
pool-1-thread-2 wrote 11 to element 1.
Next write index: 2
pool-1-thread-2 wrote 12 to element 2.
Next write index: 3
pool-1-thread-2 wrote 13 to element 3.
Next write index: 4
pool-1-thread-1 wrote  2 to element 4.
Next write index: 5
pool-1-thread-1 wrote  3 to element 5.
Next write index: 6

Contents of SimpleArray:
1 11 12 13 2 3
```

Fig. 26.10 | Class that manages an integer array to be shared by multiple threads with synchronization. (Part 3 of 3.)

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



Software Engineering Observation 26.2

Using a `ReentrantLock` *with a fairness policy avoids indefinite postponement.*

(C) 2010 Pearson Education, Inc. All rights reserved.



Performance Tip 26.4

Using a `ReentrantLock` *with a fairness policy can decrease program performance significantly.*

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



Error-Prevention Tip 26.4

When multiple threads manipulate a shared object using locks, ensure that if one thread calls method `await` *to enter the* waiting *state for a condition object, a separate thread eventually will call* `Condition` *method* `signal` *to transition the thread waiting on the condition object back to the* runnable *state. If multiple threads may be waiting on the condition object, a separate thread can call* `Condition` *method* `signalAll` *as a safeguard to ensure that all the waiting threads have another opportunity to perform their tasks. If this is not done, starvation might occur.*

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.



Error-Prevention Tip 26.5

Using interfaces `Lock` *and* `Condition` *is error prone—*`unlock` *is not guaranteed to be called, whereas the monitor in a* `synchronized` *statement will always be released when the statement completes execution.*

(C) 2010 Pearson Education, Inc. All rights reserved.



(C) 2010 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 26.22: SynchronizedBuffer.java
2  // Synchronizing access to a shared integer using the Lock and Condition
3  // interfaces
4  import java.util.concurrent.locks.Lock;
5  import java.util.concurrent.locks.ReentrantLock;
6  import java.util.concurrent.locks.Condition;
7
8  public class SynchronizedBuffer implements Buffer
9  {
10    // Lock to control synchronization with this buffer
11    private final Lock accessLock = new ReentrantLock();
12
13    // conditions to control reading and writing
14    private final Condition canWrite = accessLock.newCondition();
15    private final Condition canRead = accessLock.newCondition();
16
17    private int buffer = -1; // shared by producer and consumer threads
18    private boolean occupied = false; // whether buffer is occupied
19
20    // place int value into buffer
21    public void set( int value ) throws InterruptedException
22    {
```

Fig. 26.22 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 1 of 5.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
23          accessLock.lock(); // lock this object
24
25          // output thread information and buffer information, then wait
26          try
27          {
28             // while buffer is not empty, place thread in waiting state
29             while ( occupied )
30             {
31                System.out.println( "Producer tries to write." );
32                displayState( "Buffer full. Producer waits." );
33                canWrite.await(); // wait until buffer is empty
34             } // end while
35
36             buffer = value; // set new buffer value
37
38             // indicate producer cannot store another value
39             // until consumer retrieves current buffer value
40             occupied = true;
41
42             displayState( "Producer writes " + buffer );
43
```

Fig. 26.22 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 2 of 5.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
44             // signal thread waiting to read from buffer
45             canRead.signal();
46          } // end try
47          finally
48          {
49             accessLock.unlock(); // unlock this object
50          } // end finally
51       } // end method set
52
53       // return value from buffer
54       public int get() throws InterruptedException
55       {
56          int readValue = 0; // initialize value read from buffer
57          accessLock.lock(); // lock this object
58
59          // output thread information and buffer information, then wait
60          try
61          {
62             // if there is no data to read, place thread in waiting state
63             while ( !occupied )
64             {
65                System.out.println( "Consumer tries to read." );
```

Fig. 26.22 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 3 of 5.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
66                displayState( "Buffer empty. Consumer waits." );
67                canRead.await(); // wait until buffer is full
68             } // end while
69
70             // indicate that producer can store another value
71             // because consumer just retrieved buffer value
72             occupied = false;
73
74             readValue = buffer; // retrieve value from buffer
75             displayState( "Consumer reads " + readValue );
76
77             // signal thread waiting for buffer to be empty
78             canWrite.signal();
79          } // end try
80          finally
81          {
82             accessLock.unlock(); // unlock this object
83          } // end finally
84
85          return readValue;
86       } // end method get
87
```

Fig. 26.22 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 4 of 5.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
88      // display current operation and buffer state
89      public void displayState( String operation )
90      {
91         System.out.printf( "%-40s%d\t\t%b\n\n", operation, buffer,
92            occupied );
93      } // end method displayState
94   } // end class SynchronizedBuffer
```

Fig. 26.22 | Synchronizing access to a shared integer using the `Lock` and `Condition` interfaces. (Part 5 of 5.)

(C) 2010 Pearson Education, Inc. All rights reserved.



Error-Prevention Tip 26.6

Place calls to `Lock` *method* `unlock` *in a* `finally` *block. If an exception is thrown,* `unlock` *must still be called or deadlock could occur.*

(C) 2010 Pearson Education, Inc. All rights reserved.



Common Programming Error 26.4

Forgetting to `signal` *a waiting thread is a logic error. The thread will remain in the* waiting *state, which will prevent it from proceeding. Such waiting can lead to indefinite postponement or deadlock.*

(C) 2010 Pearson Education, Inc. All rights reserved.

```
1  // Fig. 26.23: SharedBufferTest2.java
2  // Two threads manipulating a synchronized buffer.
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
5
6  public class SharedBufferTest2
7  {
8     public static void main( String[] args )
9     {
10       // create new thread pool with two threads
11       ExecutorService application = Executors.newCachedThreadPool();
12
13       // create SynchronizedBuffer to store ints
14       Buffer sharedLocation = new SynchronizedBuffer();
15
16       System.out.printf( "%-40s%s\t\t%s\n%-40s%s\n\n", "Operation",
17          "Buffer", "Occupied", "---------", "------\t\t--------" );
18
19       // execute the Producer and Consumer tasks
20       application.execute( new Producer( sharedLocation ) );
21       application.execute( new Consumer( sharedLocation ) );
22
23       application.shutdown();
24    } // end main
25 } // end class SharedBufferTest2
```

Fig. 26.23 | Two threads manipulating a synchronized buffer. (Part 1 of 4.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
Operation                              Buffer          Occupied
---------                              ------          --------

Producer writes 1                      1               true

Producer tries to write.
Buffer full. Producer waits.           1               true

Consumer reads 1                       1               false

Producer writes 2                      2               true

Producer tries to write.
Buffer full. Producer waits.           2               true

Consumer reads 2                       2               false

Producer writes 3                      3               true

Consumer reads 3                       3               false

Producer writes 4                      4               true

Consumer reads 4                       4               false
```

Fig. 26.23 | Two threads manipulating a synchronized buffer. (Part 2 of 4.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
Consumer tries to read.
Buffer empty. Consumer waits.          4               false

Producer writes 5                      5               true

Consumer reads 5                       5               false

Consumer tries to read.
Buffer empty. Consumer waits.          5               false

Producer writes 6                      6               true

Consumer reads 6                       6               false

Producer writes 7                      7               true

Consumer reads 7                       7               false

Producer writes 8                      8               true

Consumer reads 8                       8               false

Producer writes 9                      9               true

Consumer reads 9                       9               false
```

Fig. 26.23 | Two threads manipulating a synchronized buffer. (Part 3 of 4.)

(C) 2010 Pearson Education, Inc. All rights reserved.

```
Producer writes 10                      10              true


Producer done producing
Terminating Producer
Consumer reads 10                       10              false


Consumer read values totaling 55
Terminating Consumer
```

Fig. 26.23 | Two threads manipulating a synchronized buffer. (Part 4 of 4.)

(C) 2010 Pearson Education, Inc. All rights reserved.



Thanks for listening